

CodeAstra — Context & Update Log

Purpose: A running, timestamped record of what happens on the project — decisions, changes, state snapshots. Newest entries at the top. This is the file Claude reads first to recover past + current context, and appends a new entry to each session.

Format per entry: ## YYYY-MM-DD HH:MM — <title> then bullets: *State, Changed, Decisions, Next*.

2026-09-30 — Chapter PDFs added + LLM doc discrepancy found

Changed:

- Split the manual into per-feature / per-component chapters under `project-brief/chapters/` (00 Overview, 01 M1, 02 M2, 03 M3, 04 AI Pipeline, 05 Ingestion, 06 API/Data, 07 Frontend), each rendered to its own PDF. `render.sh` now renders manual + context-log + all chapters.
- Updated `manual.md` with a chapter index and corrected LLM references.

Decisions / findings (important — corrects the older `.docs`):

- **The real LLM is NVIDIA Nemotron**, not Groq/Llama (`PROJECT_ANALYSIS.md`) and not Gemini (`AI_PIPELINE.md`) — both those docs are stale. Source of truth is `backend/src/ai/model.ts`: ChatOpenAI → `https://integrate.api.nvidia.com/v1`, model `nemotron-3-nano-30b-a3b` for M1 and `nemotron-3-super-120b-a12b` for M2 & M3, all `temperature: 0`, keyed by `NVIDIA_API_KEY`.
- **Required env is now** `PORT`, `MONGO_URI`, `GITHUB_TOKEN`, `NVIDIA_API_KEY` (config throws if missing). `GROQ_API_KEY` is no longer used.
- **New undocumented behavior:** `ai.service.ts` builds an **AST/regex-only fallback** `IAnalysisResult` (no LLM) when the LangGraph step fails, and still returns `success:true` — so the API rarely hard-500s on LLM outages.
- **Schema bug spotted:** `repoAnalysis.model.ts` stores `result.m3` as an array (`[DependencyNodeSchema]`) but the runtime `m3` is an object `{ graph, formattedAscii }` → `formattedAscii` isn't persisted. Added to `plan.md` (P3 area).

Next:

- Await user's edits on `plan.md` before code changes.
-

2026-09-30 — Baseline snapshot (full current context)

State — where the project is right now:

- **Product:** CodeAstra, an AI codebase-intelligence agent. Input a GitHub repo URL → get M1 (folder purpose), M2 (entry point + flow), M3 (dependency graph). See `manual.md`.
- **Backend:** Complete and functional. Express 5 + TS + MongoDB + LangGraph pipeline (3 parallel Groq/Llama-3.3 nodes → combine). Caching via `SHA256(repoUrl)`. Single endpoint `POST /api/analysis`. Default port **4000**.
- **Frontend:** React 19 + Vite + Tailwind SPA. Restructured into a 4-layer `features/` layout (`app/`, `shared/`, `features/analysis|chat|code/`). Core analysis pages consume real M1/M2/M3 data via `AnalysisContext`; chat, code-IDE, architecture pages are static mocks.
- **Git:** on `main`, working tree clean. Recent history includes the `features/` restructure (`18f4c67`, `595b5f9`) and merges from `AI-layer`. An `AI-layer` branch is ahead on origin.
- **Docs:** `.docs/` holds the detailed reference set — `PROJECT_ANALYSIS.md` (backend, 442 lines), `FRONTEND_FLOW_AND_IMPROVEMENTS.md` (pre-restructure paths), `AI_PIPELINE.md`, `restructure.md`, `AI_LAYER_*`, `authentication.md`, `depolyment.md`, `ERROR.md`, `feature.md`, + 3 architecture PNGs.

Known open issues (carried forward — see `manual.md §8` and `plan.md`):

1. Frontend API base URL is inconsistent: context posts to `:5000`, `LandingPage` has a leftover `:4000` axios call, backend is `4000`. One of the calls also sends a raw string instead of `{ repoUrl }`.
2. `AnalysisContext.error` is never set → failures silently show mock data; no `isLive` flag.
3. `transformAnalysisResult` uses `Math.random()` → unstable dashboard numbers.
4. `LoadingPage analyzeRepo` in `useEffect` → `StrictMode` double-fetch.
5. Restructure incomplete: `hooks/useAnalysis.ts` and `services/analysis.api.ts` are empty; fetch+transform still inside context.
6. Backend hardening: no tests, no `repoUrl` validation, no auth/rate-limit, open CORS, hardcoded limits, `console.log` logging.
7. Docs drift: `FRONTEND_FLOW_AND_IMPROVEMENTS.md` still references the old `src/pages`, `src/components` paths.

Changed this session:

- Created `project-brief/` with `manual.md`, `context-log.md` (this file), `plan.md`, and a `render.sh` to produce `manual.pdf` + `context-log.pdf`.

Decisions:

- Source of truth is markdown; PDFs are rendered artifacts (via `marked` + Chrome headless). Rationale: PDFs can't be appended to and aren't easily machine-readable for context recovery.

Next:

- Await user's edits/notes on `plan.md` before making any code changes.
-